

COMPTE RENDU

Administration Reseaux et Programmation Systeme – TP4 – Installation et
Configuration d'un serveur DNS
3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignant : M.Bakhouya & M.Aqachtoul
Date : 12 avril 2026

Table des matières

1	Introduction	2
1.1	Objectifs du TP	2
1.2	Contexte et prérequis	2
1.3	Rappel théorique DNS	2
2	Topologie réseau et architecture DNS	3
2.1	Topologie générale	3
2.2	Table d'adressage complète	3
2.3	Table de routage des namespaces	4
3	Partie 1 – Déploiement du serveur DNS (BIND9)	4
3.1	Vérification du réseau de base (TP0)	4
3.2	Création du namespace DNS et connexion au GATEWAY	4
3.3	Configuration IP – côté DNS et GATEWAY	5
3.4	Connexion directe namespace DNS ↔ hôte (accès Internet)	5
4	Partie 2 – Configuration BIND9 et zones DNS	6
4.1	Zone directe – db.lab.local	6
4.2	Zone inverse – db.192.168.30	6
4.3	Déclaration des zones – named.conf.local	7
4.4	Options BIND – named.conf.options	7
5	Partie 3 – Déploiement complet et configuration clients	8
5.1	Lancement du déploiement – sortie terminal complète	8
5.2	Configuration des résolveurs DNS sur les clients	9
5.3	Routes vers le réseau DNS depuis les clients	10
6	Vérification et tests DNS	10
6.1	Test de connectivité PC → serveur DNS	10
6.2	Test de résolution DNS locale (zone lab.local)	10
6.3	Test de résolution inverse (zone 30.168.192.in-addr.arpa)	11
6.4	Test de résolution externe (forwarding vers 8.8.8.8)	11
6.5	Vérification du routage dans chaque namespace	11
6.6	Capture tcpdump – Échange DNS	12
6.7	Vérification de la configuration BIND	13
7	Scripts Bash développés	13
7.1	Vue d'ensemble	13
7.2	Ordre d'exécution	13
7.3	Points techniques clés	14
8	Résultats	15
8.1	Tableau récapitulatif des tests	15
8.2	Résumé des enregistrements DNS configurés	15
	Conclusion	15

1 Introduction

1.1 Objectifs du TP

Ce TP porte sur le déploiement et la configuration d'un serveur DNS (Domain Name Service) dans un environnement réseau virtuel basé sur les namespaces Linux. Il s'appuie sur la topologie de base du TP0 (IPs statiques) et la complète avec un serveur de noms autoritaire pour le domaine `lab.local`. Les objectifs sont les suivants :

- Comprendre le rôle du DNS dans la correspondance entre noms d'hôtes et adresses IP.
- Déployer un serveur BIND9 dans un namespace réseau isolé (DNS).
- Configurer les zones directe et inverse pour le domaine `lab.local`.
- Vérifier la résolution de noms locaux (`PC1.lab.local`, `PC2.lab.local`) depuis les namespaces clients.
- Étendre le serveur DNS pour résoudre des noms externes (ex. `google.com`) via un forwarder vers `8.8.8.8`, accessible grâce à une veth directe entre le namespace DNS et l'hôte.

1.2 Contexte et prérequis

Ce TP s'appuie sur la topologie construite en TP0 : deux LANs privés (`192.168.10.0/24` et `192.168.20.0/24`), une passerelle (GATEWAY) et un FAI simulé (ISP). Le serveur DNS est déployé dans un namespace dédié (DNS) relié au GATEWAY via une paire veth, formant un troisième réseau (`192.168.30.0/24`).

1.3 Rappel théorique DNS

Le DNS (Domain Name System) est un service Internet qui traduit les noms d'hôtes lisibles par l'humain en adresses IP exploitables par les machines. Un serveur DNS autoritaire maintient les enregistrements d'une zone et répond aux requêtes de résolution.

Type d'enregistrement	Sens	Description
A	Nom → IP	Résolution directe (hostname → adresse IPv4)
PTR	IP → Nom	Résolution inverse (adresse IPv4 → hostname)
NS	Zone → Serveur	Déclare le serveur de noms autoritaire
SOA	Métadonnées zone	Paramètres de contrôle de la zone

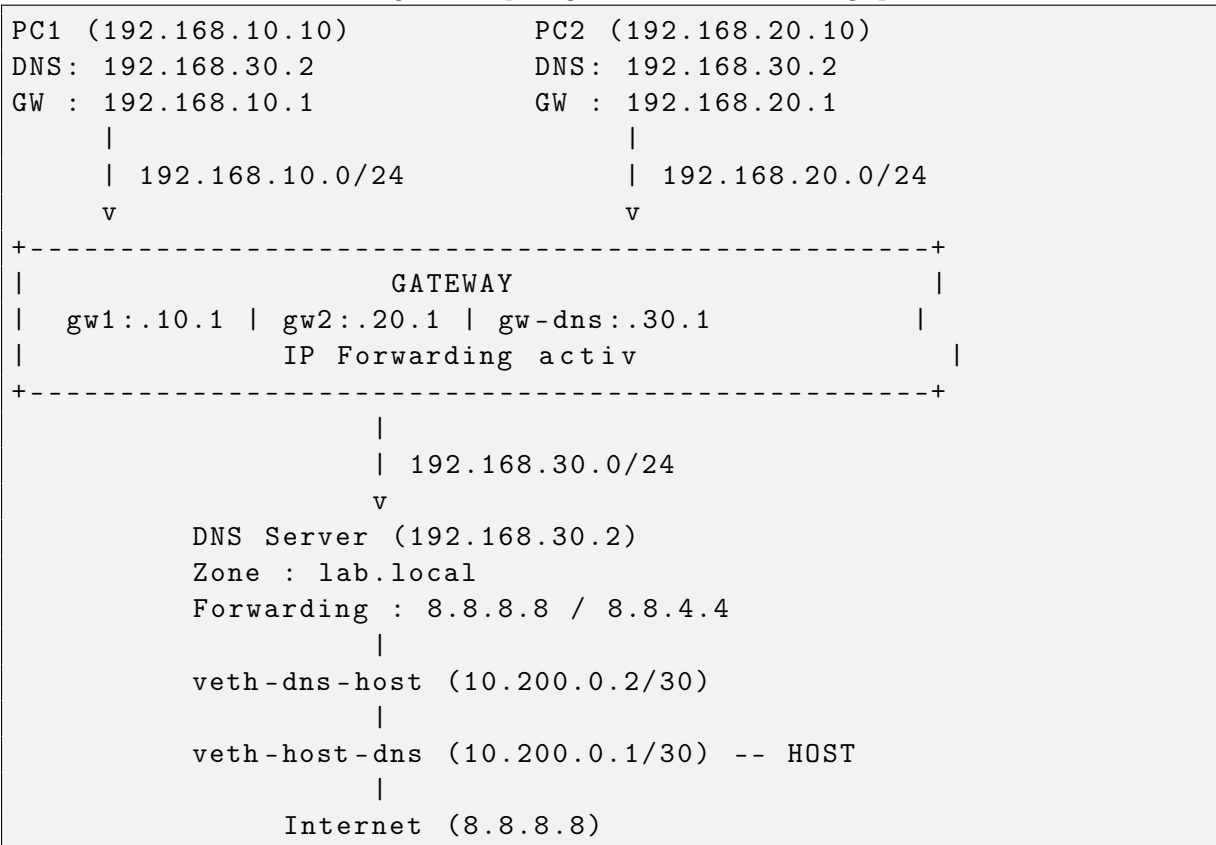
Les namespaces Linux permettent d'émuler des hôtes et serveurs isolés sur une seule machine physique : chaque namespace dispose de sa propre pile réseau, ses propres interfaces, routes et règles de pare-feu, indépendamment des autres.

2 Topologie réseau et architecture DNS

2.1 Topologie générale

La topologie étend celle du TP0 en ajoutant un namespace DNS connecté au GATEWAY via une paire veth (`veth-dns` ↔ `gw-dns`). Une seconde paire veth (`veth-host-dns` ↔ `veth-dns-host`) relie directement le namespace DNS à l'hôte pour l'accès Internet.

Listing 1 – Topologie TP4 – Schéma logique



2.2 Table d'adressage complète

Équipement	Interface	Adresse IP	Réseau
PC1	veth1	192.168.10.10/24	LAN1
PC2	veth2	192.168.20.10/24	LAN2
GATEWAY	gw1	192.168.10.1/24	LAN1
GATEWAY	gw2	192.168.20.1/24	LAN2
GATEWAY	veth3	209.165.200.226/27	WAN
GATEWAY	gw-dns	192.168.30.1/24	DNS Network
ISP	isp1	209.165.200.225/27	WAN
DNS	veth-dns	192.168.30.2/24	DNS Network
DNS	veth-dns-host	10.200.0.2/30	DNS ↔ Hôte
Hôte	veth-host-dns	10.200.0.1/30	DNS ↔ Hôte

2.3 Table de routage des namespaces

Namespace	Destination	Via
PC1	192.168.30.0/24 (DNS Network)	192.168.10.1 (GATEWAY)
PC1	default	192.168.10.1 (GATEWAY)
PC2	192.168.30.0/24 (DNS Network)	192.168.20.1 (GATEWAY)
PC2	default	192.168.20.1 (GATEWAY)
DNS	192.168.10.0/24 (LAN1)	192.168.30.1 (GATEWAY)
DNS	192.168.20.0/24 (LAN2)	192.168.30.1 (GATEWAY)
DNS	default	10.200.0.1 (Hôte)

3 Partie 1 – Déploiement du serveur DNS (BIND9)

3.1 Vérification du réseau de base (TP0)

Avant de déployer le DNS, le réseau TP0 doit être opérationnel. Le script `enable_dns.sh` vérifie la présence des namespaces PC1, PC2, GATEWAY et ISP.

Listing 2 – Vérification des prérequis TP0 – sortie terminal

```
1 hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau
  /TP4$
2 ./setup_network.sh
3 [INFO] Nettoyage pr ventif...
4 [OK] Nettoyage termin
5 [INFO] === TAPE 1 : Cr ation des namespaces ===
6 [OK] Namespace PC1 cr
7 [OK] Namespace PC2 cr
8 [OK] Namespace GATEWAY cr
9 [OK] Namespace ISP cr
10 [INFO] === TAPE 4 : Adressage IP et activation des interfaces ===
11 [OK] PC1 : veth1 = 192.168.10.10/24 [UP]
12 [OK] PC2 : veth2 = 192.168.20.10/24 [UP]
13 [OK] GATEWAY : gw1=192.168.10.1/24 | gw2=192.168.20.1/24 |
14 veth3=209.165.200.226/27 [UP]
15 [OK] ISP : isp1 = 209.165.200.225/27 [UP]
16 =====
17 R seau virtuel d ploy avec succ s !
18 =====
```

3.2 Création du namespace DNS et connexion au GATEWAY

Listing 3 – Création namespace DNS et paire veth

```
1 # Cr ation du namespace DNS avec loopback
2 sudo ip netns add DNS
3 sudo ip netns exec DNS ip link set lo up
4
5 # Paire veth : veth-dns (DNS) <-> gw-dns (GATEWAY)
6 sudo ip link add veth-dns type veth peer name gw-dns
7 sudo ip link set veth-dns netns DNS
```

```
8 sudo ip link set gw-dns netns GATEWAY
```

3.3 Configuration IP – côté DNS et GATEWAY

Listing 4 – Adressage IP du namespace DNS et du GATEWAY

```
1 # C t DNS
2 sudo ip netns exec DNS ip addr add 192.168.30.2/24 dev veth-dns
3 sudo ip netns exec DNS ip link set veth-dns up
4 sudo ip netns exec DNS ip route add default via 192.168.30.1
5
6 # C t GATEWAY
7 sudo ip netns exec GATEWAY ip addr add 192.168.30.1/24 dev gw-dns
8 sudo ip netns exec GATEWAY ip link set gw-dns up
9 sudo ip netns exec GATEWAY sysctl -w net.ipv4.ip_forward=1
```

3.4 Connexion directe namespace DNS ↔ hôte (accès Internet)

L'ISP étant un namespace simulé sans accès Internet réel, une veth directe entre le namespace DNS et l'hôte est nécessaire pour que BIND9 puisse forwarder les requêtes externes vers 8.8.8.8.

Listing 5 – Création de la veth hôte et MASQUERADE

```
1 # Paire veth : veth-host-dns (h te) <-> veth-dns-host (DNS)
2 sudo ip link add veth-host-dns type veth peer name veth-dns-host
3 sudo ip link set veth-dns-host netns DNS
4
5 # C t h te
6 sudo ip addr add 10.200.0.1/30 dev veth-host-dns
7 sudo ip link set veth-host-dns up
8
9 # C t namespace DNS
10 sudo ip netns exec DNS ip addr add 10.200.0.2/30 dev veth-dns-host
11 sudo ip netns exec DNS ip link set veth-dns-host up
12
13 # Route par défaut DNS via h te (pour joindre Internet)
14 sudo ip netns exec DNS ip route replace default via 10.200.0.1
15
16 # Routes retour : réponses DNS -> LANs via GATEWAY
17 sudo ip netns exec DNS ip route add 192.168.10.0/24 via
18     192.168.30.1
19 sudo ip netns exec DNS ip route add 192.168.20.0/24 via
20     192.168.30.1
21
22 # MASQUERADE sur l'h te : namespace DNS sort avec l'IP réelle
23 IFACE_SORTIE=$(ip route get 8.8.8.8 | grep -oP 'dev \K\S+' | head
    -1)
24 sudo iptables -t nat -A POSTROUTING \
25     -s 10.200.0.2/32 -o "$IFACE_SORTIE" -j MASQUERADE
```

4 Partie 2 – Configuration BIND9 et zones DNS

4.1 Zone directe – db.lab.local

Le fichier de zone directe définit les enregistrements A permettant la résolution hostname → IP pour le domaine lab.local.

Listing 6 – Fichier de zone directe /etc/bind/db.lab.local

```
1 $TTL 604800
2 @ IN SOA ns.lab.local. admin.lab.local. (
3     20260101      ; Serial   (YYYYMMDD)
4     604800        ; Refresh  (7 jours)
5     86400         ; Retry   (1 jour)
6     2419200       ; Expire   (28 jours)
7     604800 )      ; Minimum TTL (7 jours)
8
9 ; Serveur de noms autoritaire pour lab.local
10 @ IN NS ns.lab.local.
11
12 ; Enregistrements A (hostname -> IP)
13 ns IN A 192.168.30.2 ; ns.lab.local -> 192.168.30.2
14 PC1 IN A 192.168.10.10 ; PC1.lab.local -> 192.168.10.10
15 PC2 IN A 192.168.20.10 ; PC2.lab.local -> 192.168.20.10
```

L'enregistrement SOA (Start of Authority) déclare ce serveur DNS (ns.lab.local.) comme autoritaire pour la zone lab.local et contient les paramètres de contrôle de zone (TTL, délais de refresh, numéro de série).

4.2 Zone inverse – db.192.168.30

Le fichier de zone inverse permet la résolution IP → hostname (enregistrement PTR). Il est utilisé par BIND pour mapper une adresse IP à son nom d'hôte correspondant.

Listing 7 – Fichier de zone inverse /etc/bind/db.192.168.30

```
1 $TTL 604800
2 @ IN SOA ns.lab.local. admin.lab.local. (
3     20260101      ; Serial
4     604800        ; Refresh
5     86400         ; Retry
6     2419200       ; Expire
7     604800 )      ; Minimum TTL
8
9 ; Serveur de noms responsable des lookups inverses
10 @ IN NS ns.lab.local.
11
12 ; Enregistrement PTR : 192.168.30.2 -> ns.lab.local
13 ; "2" est le dernier octet de 192.168.30.2
14 ; Utilis pour la r solution inverse (IP -> nom)
15 2 IN PTR ns.lab.local.
```

Explication de la ligne 2 `IN PTR ns.lab.local.` : Dans la zone inverse `30.168.192.in-addr.arpa`, le chiffre 2 représente le dernier octet de l'IP `192.168.30.2`. Cette entrée PTR (Pointer Record) établit la correspondance inverse : quand un client demande « quel est le nom de `192.168.30.2` ? », BIND répond `ns.lab.local.`. C'est l'opposé exact d'un enregistrement A.

4.3 Déclaration des zones – `named.conf.local`

Listing 8 – Déclaration des zones dans `named.conf.local`

```
1 // Zone directe : r solution hostname -> IP
2 zone "lab.local" {
3     type master;
4     file "/etc/bind/db.lab.local";
5 };
6
7 // Zone inverse : r solution IP -> hostname pour 192.168.30.0/24
8 zone "30.168.192.in-addr.arpa" {
9     type master;
10    file "/etc/bind/db.192.168.30";
11 };
```

Ce fichier ne contient pas de records DNS – il indique uniquement à BIND quelles zones il gère et où se trouvent les fichiers de données correspondants.

4.4 Options BIND – `named.conf.options`

Listing 9 – Options BIND9 avec forwarding externe

```
1 options {
2     directory "/var/cache/bind";
3
4     // R cursivit : BIND peut interroger des DNS externes
5     recursion yes;
6
7     // Accepter les requ tes depuis n'importe quelle IP (lab)
8     allow-query { any; };
9
10    // couter uniquement sur l'interface du namespace DNS
11    listen-on { 192.168.30.2; };
12
13    // Forcer BIND utiliser 10.200.0.2 comme IP source
14    // pour les requ tes sortantes vers les forwarders
15    // (sinon BIND utilise 192.168.30.2, non routable sur Internet)
16    query-source address 10.200.0.2;
17
18    // Forwarders : si domaine non local, d l guer Google DNS
19    forwarders {
20        8.8.8.8;
21        8.8.4.4;
22    };
```

```

23
24     dnssec-validation no;
25 };

```

5 Partie 3 – Déploiement complet et configuration clients

5.1 Lancement du déploiement – sortie terminal complète

Listing 10 – Exécution de enable_dns.sh – sortie terminal

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau
/TP4$
./enable_dns.sh
[INFO]   V r i f i c a t i o n   q u e   l e   r   s e a u   T P 0   e s t   b i e n   d   p l o y   . . .
[OK]     R   s e a u   T P 0   d   t e c t

[INFO]   ===   T A P E   1   :   N e t t o y a g e   p r   v e n t i f   D N S   ===
[OK]     N e t t o y a g e   D N S   t e r m i n

[INFO]   ===   T A P E   2   :   C r   a t i o n   d u   n a m e s p a c e   D N S   ===
[OK]     N a m e s p a c e   D N S   c r       ( l o o p b a c k   U P )

[INFO]   ===   T A P E   3   :   C r   a t i o n   d u   l i e n   v i r t u e l   D N S   <->   G A T E W A Y
===
[OK]     v e t h - d n s   <->   g w - d n s   ( D N S   <->   G A T E W A Y )
[OK]     v e t h - d n s   ->   n a m e s p a c e   D N S
[OK]     g w - d n s       ->   n a m e s p a c e   G A T E W A Y

[INFO]   ===   T A P E   4   :   C o n f i g u r a t i o n   I P   --   c   t   D N S   ===
[OK]     D N S       :   v e t h - d n s   =   1 9 2 . 1 6 8 . 3 0 . 2 / 2 4   [ U P ]
[OK]     D N S       :   r o u t e   p a r   d   f a u t   v i a   1 9 2 . 1 6 8 . 3 0 . 1   ( G A T E W A Y )

[INFO]   ===   T A P E   5   :   C o n f i g u r a t i o n   I P   --   c   t   G A T E W A Y   ===
net.ipv4.ip_forward = 1
[OK]     G A T E W A Y   :   g w - d n s   =   1 9 2 . 1 6 8 . 3 0 . 1 / 2 4   [ U P ]
[OK]     G A T E W A Y   :   I P   f o r w a r d i n g   a c t i v

[INFO]   ===   T A P E   5 b   :   C o n n e x i o n   d i r e c t e   n a m e s p a c e   D N S   <->   h   t e
===
[OK]     v e t h - h o s t - d n s   <->   v e t h - d n s - h o s t   c r   e s
[OK]     v e t h - d n s - h o s t   ->   n a m e s p a c e   D N S
[OK]     H   t e   :   v e t h - h o s t - d n s   =   1 0 . 2 0 0 . 0 . 1 / 3 0   [ U P ]
[OK]     D N S   :   v e t h - d n s - h o s t   =   1 0 . 2 0 0 . 0 . 2 / 3 0   [ U P ]
[OK]     D N S   :   r o u t e   p a r   d   f a u t   ->   1 0 . 2 0 0 . 0 . 1   ( h   t e )
[OK]     D N S   :   r o u t e   1 9 2 . 1 6 8 . 1 0 . 0 / 2 4   v i a   1 9 2 . 1 6 8 . 3 0 . 1   ( L A N 1   ->
G A T E W A Y )
[OK]     D N S   :   r o u t e   1 9 2 . 1 6 8 . 2 0 . 0 / 2 4   v i a   1 9 2 . 1 6 8 . 3 0 . 1   ( L A N 2   ->
G A T E W A Y )
[OK]     M A S Q U E R A D E   h   t e   :   1 0 . 2 0 0 . 0 . 2   ->   w l p 0 s 2 0 f 3   ( I n t e r n e t )

```

```

[INFO]    ===  TAPE  6 : V r i f i c a t i o n / I n s t a l l a t i o n d e B I N D 9  ===
[OK]      BIND9 disponible : BIND 9.18.39-0ubuntu0.24.04.3-Ubuntu

[INFO]    ===  TAPE  7 : C r a t i o n d e s f i c h i e r s d e z o n e B I N D  ===
[OK]      Zone directe cr e : /etc/bind/db.lab.local
[OK]      Zone inverse cr e : /etc/bind/db.192.168.30

[INFO]    ===  TAPE  8 : D c l a r a t i o n d e s z o n e s d a n s n a m e d . c o n f .
          local ===
[OK]      named.conf.local configur

[INFO]    ===  TAPE  9 : C o n f i g u r a t i o n d e s o p t i o n s B I N D  ===
[OK]      named.conf.options configur (forwarding vers 8.8.8.8)

[INFO]    ===  TAPE 10 : V a l i d a t i o n d e l a c o n f i g u r a t i o n B I N D  ===
[OK]      named-checkconf : configuration valide
zone lab.local/IN: loaded serial 20260101
OK
[OK]      named-checkzone lab.local : zone valide

[INFO]    ===  TAPE 11 : D m a r r a g e d u s e r v e u r D N S ( n a m e d )  ===
[OK]      named en cours d' e x c u t i o n d a n s l e n a m e s p a c e D N S

[INFO]    ===  TAPE 12 : C o n f i g u r a t i o n d e s r s o l v e u r s D N S s u r P C 1
          et PC2 ===
[OK]      PC1 : resolv.conf -> 192.168.30.2 (primaire) | 8.8.8.8 (
          fallback)
[OK]      PC2 : resolv.conf -> 192.168.30.2 (primaire) | 8.8.8.8 (
          fallback)

[INFO]    ===  TAPE 13 : A j o u t d e s r o u t e s v e r s l e r s e a u D N S  ===
[OK]      PC1 : route 192.168.30.0/24 via 192.168.10.1
[OK]      PC2 : route 192.168.30.0/24 via 192.168.20.1

=====
                S e r v e u r  D N S  T P 4  d  p l o y  a v e c  s u c c  s  !
=====

```

5.2 Configuration des résolveurs DNS sur les clients

Listing 11 – Configuration /etc/resolv.conf sur PC1 et PC2

```

1 # PC1 et PC2 : r s o l v e u r p r i m a i r e = D N S l a b , f a l l b a c k = G o o g l e D N S
2 sudo ip netns exec PC1 bash -c \
3     'echo "nameserver 192.168.30.2" > /etc/resolv.conf && \
4     echo "nameserver 8.8.8.8" >> /etc/resolv.conf'
5 sudo ip netns exec PC2 bash -c \
6     'echo "nameserver 192.168.30.2" > /etc/resolv.conf && \
7     echo "nameserver 8.8.8.8" >> /etc/resolv.conf'

```

5.3 Routes vers le réseau DNS depuis les clients

Listing 12 – Ajout des routes vers 192.168.30.0/24 sur PC1 et PC2

```
1 # PC1 : pour atteindre 192.168.30.0/24, passer par 192.168.10.1
2 sudo ip netns exec PC1 ip route replace 192.168.30.0/24 via
   192.168.10.1
3
4 # PC2 : pour atteindre 192.168.30.0/24, passer par 192.168.20.1
5 sudo ip netns exec PC2 ip route replace 192.168.30.0/24 via
   192.168.20.1
```

6 Vérification et tests DNS

6.1 Test de connectivité PC → serveur DNS

Listing 13 – Ping de PC1 et PC2 vers le serveur DNS

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau
/TP4$
sudo ip netns exec PC1 ping -c 1 192.168.30.2
PING 192.168.30.2 (192.168.30.2) 56(84) bytes of data.
64 bytes from 192.168.30.2: icmp_seq=1 ttl=63 time=0.061 ms

--- 192.168.30.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.061/0.061/0.061/0.000 ms

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau
/TP4$
sudo ip netns exec PC2 ping -c 1 192.168.30.2
PING 192.168.30.2 (192.168.30.2) 56(84) bytes of data.
64 bytes from 192.168.30.2: icmp_seq=1 ttl=63 time=0.086 ms

--- 192.168.30.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.086/0.086/0.086/0.000 ms
```

6.2 Test de résolution DNS locale (zone lab.local)

Résolution directe – PC1 → PC2.lab.local

Listing 14 – Résolution directe depuis PC1 – dig PC2.lab.local

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau
/TP4$
sudo ip netns exec PC1 dig PC2.lab.local +short
192.168.20.10
```

Résolution directe – PC2 → PC1.lab.local

Listing 15 – Résolution directe depuis PC2 – dig PC1.lab.local

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau/TP4$  
sudo ip netns exec PC2 dig PC1.lab.local +short  
192.168.10.10
```

6.3 Test de résolution inverse (zone 30.168.192.in-addr.arpa)

Listing 16 – Résolution inverse – dig -x 192.168.30.2

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau/TP4$  
sudo ip netns exec PC1 dig -x 192.168.30.2 +short  
ns.lab.local.
```

La résolution inverse retourne bien `ns.lab.local.`, confirmant que l'enregistrement PTR 2 IN PTR `ns.lab.local.` est correctement configuré dans la zone inverse.

6.4 Test de résolution externe (forwarding vers 8.8.8.8)

Listing 17 – Résolution d'un nom externe – dig google.com

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau/TP4$  
sudo ip netns exec PC1 dig google.com +short  
216.58.205.78  
  
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau/TP4$  
sudo ip netns exec PC2 dig google.com +short  
216.58.205.78
```

PC1 envoie la requête à 192.168.30.2. BIND vérifie ses zones locales, ne trouve pas `google.com`, puis forward vers 8.8.8.8. La réponse est mise en cache et retournée à PC1. Le cheminement est le suivant :

Listing 18 – Cheminement d'une requête externe

```
PC1 -> 192.168.30.2 (BIND v r i f i e  l a b . l o c a l ) -> n o n  t r o u v  
-> BIND forward via 10.200.0.2 -> H t e (MASQUERADE)  
-> wlp0s20f3 (IP r e l l e ) -> 8.8.8.8  
-> R p o n s e : 216.58.205.78  
-> BIND cache et retourne PC1
```

6.5 Vérification du routage dans chaque namespace

Listing 19 – Tables de routage PC1, PC2 et DNS

```
hathouti@Taha-inspiron-16:~$ sudo ip netns exec PC1 ip route
default via 192.168.10.1 dev veth1
192.168.10.0/24 dev veth1 proto kernel scope link src 192.168.10.10
192.168.30.0/24 via 192.168.10.1 dev veth1

hathouti@Taha-inspiron-16:~$ sudo ip netns exec PC2 ip route
default via 192.168.20.1 dev veth2
192.168.20.0/24 dev veth2 proto kernel scope link src 192.168.20.10
192.168.30.0/24 via 192.168.20.1 dev veth2

hathouti@Taha-inspiron-16:~$ sudo ip netns exec DNS ip route
default via 10.200.0.1 dev veth-dns-host
10.200.0.0/30 dev veth-dns-host proto kernel scope link src
    10.200.0.2
192.168.10.0/24 via 192.168.30.1 dev veth-dns
192.168.20.0/24 via 192.168.30.1 dev veth-dns
192.168.30.0/24 dev veth-dns proto kernel scope link src
    192.168.30.2
```

Les routes retour vers LAN1 et LAN2 via 192.168.30.1 (GATEWAY) sont essentielles : sans elles, les réponses DNS partiraient par la veth hôte (route par défaut) et PC1/PC2 ne recevraient jamais les réponses.

6.6 Capture tcpdump – Échange DNS

Un tcpdump est exécuté sur l'interface veth-dns du namespace DNS pendant une requête dig PC2.lab.local depuis PC1, confirmant l'échange requête/réponse DNS.

Listing 20 – Capture tcpdump sur veth-dns – échange DNS

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau
/TP4$
sudo ip netns exec DNS tcpdump -i veth-dns -n port 53 &

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/Admin_Reseau
/TP4$
sudo ip netns exec PC1 dig PC2.lab.local +short
192.168.20.10

tcpdump: verbose output suppressed, use -v[v]... for full protocol
decode
listening on veth-dns, link-type EN10MB (Ethernet), snapshot length
262144 bytes
21:49:57.587502 IP 192.168.10.10.51767 > 192.168.30.2.53:
    47197+ [1au] A? PC2.lab.local. (54)
21:49:57.587723 IP 192.168.30.2.53 > 192.168.10.10.51767:
    47197* 1/0/1 A 192.168.20.10 (86)
2 packets captured
2 packets received by filter
0 packets dropped by kernel
```

Analyse de la capture :

- **Paquet 1 (requête)** : PC1 (192.168.10.10, port 51767) envoie une requête DNS de type A pour PC2.lab.local au serveur DNS (192.168.30.2, port 53). Le flag + indique que la récursivité est demandée.
- **Paquet 2 (réponse)** : Le serveur DNS répond immédiatement avec l'enregistrement A 192.168.20.10. Le flag * confirme que la réponse est autoritaire (le DNS possède la zone).

6.7 Vérification de la configuration BIND

Listing 21 – Validation syntaxique de la configuration BIND

```
hathouti@Taha-inspiron-16:~$ sudo ip netns exec DNS named-checkconf
# (aucune sortie = configuration valide)

hathouti@Taha-inspiron-16:~$ sudo ip netns exec DNS \
    named-checkzone lab.local /etc/bind/db.lab.local
zone lab.local/IN: loaded serial 20260101
OK
```

7 Scripts Bash développés

7.1 Vue d'ensemble

Script	Rôle
setup_network.sh	Déploie la topologie TP0 (PC1, PC2, GATEWAY, ISP)
cleanup_network.sh	Supprime tous les namespaces TP0
setup_dhcp.sh	Ajoute DHCP_SERVER avec dnsmasq (TP2)
cleanup_dhcp.sh	Supprime le DHCP, préserve TP0
enable_static_nat.sh	Configure SNAT 1 :1 sur GATEWAY (TP3)
cleanup_static_nat.sh	Vide les règles SNAT
enable_dynamic_nat.sh	Configure MASQUERADE/PAT sur GATEWAY (TP3)
cleanup_dynamic_nat.sh	Vide les règles MASQUERADE
enable_dns.sh	Déploie le namespace DNS + BIND9 (TP4, 13 étapes)
clean_dns.sh	Supprime le DNS, préserve TP0

7.2 Ordre d'exécution

Listing 22 – Séquence d'utilisation complète TP0 → TP4

```
1 # 1. Réseau de base (TP0)
2 bash setup_network.sh
3
4 # 2. NAT Dynamique pour l'acc s Internet (TP3 -- requis par TP4
   Partie 2)
5 bash enable_dynamic_nat.sh
```

```

6
7 # 3. Serveur DNS (TP4)
8 bash enable_dns.sh
9
10 # Tests
11 sudo ip netns exec PC1 ping -c 1 192.168.30.2
12 sudo ip netns exec PC1 dig PC2.lab.local +short
13 sudo ip netns exec PC2 dig PC1.lab.local +short
14 sudo ip netns exec PC1 dig -x 192.168.30.2 +short
15 sudo ip netns exec PC1 dig google.com +short
16
17 # Nettoyage DNS seul (rseau TP0 intact)
18 bash clean_dns.sh
19
20 # Nettoyage total
21 bash cleanup_network.sh

```

7.3 Points techniques clés

Problème de routing asymétrique – solution par routes spécifiques

Le namespace DNS dispose de deux routes de sortie : la veth hôte (veth-dns-host, route par défaut) et la veth GATEWAY (veth-dns, 192.168.30.0/24). Sans routes spécifiques, les réponses DNS vers LAN1/LAN2 partiraient par la veth hôte, rendant PC1 et PC2 inaccessibles depuis le DNS.

Listing 23 – Routes spécifiques pour éviter le routage asymétrique

```

1 # Forcer les rponses vers LAN1 et LAN2 passer par le GATEWAY
2 sudo ip netns exec DNS ip route add 192.168.10.0/24 via
   192.168.30.1
3 sudo ip netns exec DNS ip route add 192.168.20.0/24 via
   192.168.30.1

```

Problème d'IP source BIND – solution par query-source

BIND écoute sur 192.168.30.2 mais doit contacter 8.8.8.8 via veth-dns-host (10.200.0.2). Sans query-source, BIND utilise 192.168.30.2 comme IP source pour les requêtes sortantes, IP non routable sur Internet. La directive query-source force l'utilisation de 10.200.0.2.

Listing 24 – Directive query-source dans named.conf.options

```

1 options {
2     ...
3     listen-on      { 192.168.30.2; };    // écoute sur cette IP
4     query-source address 10.200.0.2;    // Source pour requêtes
       sortantes
5     forwarders     { 8.8.8.8; 8.8.4.4; };
6     ...
7 };

```

8 Résultats

8.1 Tableau récapitulatif des tests

Test	Description	Résultat
1	Ping PC1 → 192.168.30.2 (serveur DNS)	SUCCÈS
2	Ping PC2 → 192.168.30.2 (serveur DNS)	SUCCÈS
3	dig PC2.lab.local depuis PC1	192.168.20.10
4	dig PC1.lab.local depuis PC2	192.168.10.10
5	dig -x 192.168.30.2 (résolution inverse)	ns.lab.local.
6	dig google.com depuis PC1 (forwarding)	216.58.205.78
7	dig google.com depuis PC2 (forwarding)	216.58.205.78
8	named-checkconf – validation BIND	VALIDE
9	named-checkzone lab.local – zone valide	OK
10	tcpdump : échange requête/réponse DNS capturé	2 paquets

8.2 Résumé des enregistrements DNS configurés

Nom	Type	Valeur	Zone
ns.lab.local.	A	192.168.30.2	lab.local
PC1.lab.local.	A	192.168.10.10	lab.local
PC2.lab.local.	A	192.168.20.10	lab.local
2.30.168.192.in-addr.arpa.	PTR	ns.lab.local.	Zone inverse

Conclusion

Ce TP4 a permis de déployer et valider un serveur DNS autoritaire complet dans un environnement réseau virtuel basé sur des namespaces Linux. Le serveur BIND9, déployé dans le namespace isolé `DNS`, gère la zone `lab.local` et assure la résolution de noms locaux pour PC1 et PC2.

La résolution interne fonctionne dans les deux sens : directe (A records : `PC1.lab.local` → 192.168.10.10) et inverse (PTR record : 192.168.30.2 → `ns.lab.local.`). La résolution externe a nécessité une architecture supplémentaire : une veth directe entre le namespace `DNS` et l'hôte, avec `MASQUERADE`, permettant à BIND de contacter 8.8.8.8 et de forwarder les requêtes non locales.

Deux problèmes techniques ont été résolus : le routage asymétrique (réponses DNS qui partaient par la mauvaise interface) corrigé par des routes spécifiques vers LAN1 et LAN2, et le problème d'IP source BIND corrigé par la directive `query-source`.

L'approche par namespaces Linux démontre sa puissance pour émuler des architectures réseau complexes sur une seule machine : chaque namespace se comporte comme un

hôte isolé avec sa propre pile réseau, permettant de tester des configurations de production (DNS, NAT, DHCP) de manière reproductible et sans risque pour l'infrastructure physique.